

Þetta lítur mjög vel út. Förum í næsta skref — Technical Specification v0.5

Frábært. Þá förum við yfir í **v0.5**, þar sem hönnunin verður orðin nægilega nákvæm til að hægt sé að nota hana sem grunn fyrir fyrstu raunverulegu implementation.

Technical Specification v0.5

Production PostgreSQL Schema & Migrations

Project: Icelandic Legal Case Management System — ILCMS

Version: 0.5

Status: Implementation Specification

Database: PostgreSQL 16+

API: REST / JSON

Schema strategy: Versioned migrations

Primary key: UUID

Time: UTC / TIMESTAMPTZ

1. Markmið v0.5

Í v0.3 skilgreindum við gagnagrunninn og API-ið.

Í v0.4 skilgreindum við:

- lifecycle
- workflow
- permissions
- RBAC
- case ACL
- audit
- legal holds
- retention
- separation of duties

Í v0.5 sameinum við þetta í **framkvæmanlegt PostgreSQL schema**.

Markmiðið er að hægt sé að gera:

PostgreSQL

↓

Run migrations

↓
Empty production database
↓
Ready for API

2. Migration structure

Ég mæli með:

```
db/  
└─ migrations/  
    └─ 001_extensions.sql  
    └─ 002_users.sql  
    └─ 003_cases.sql  
    └─ 004_parties.sql  
    └─ 005_documents.sql  
    └─ 006_evidence.sql  
    └─ 007_legal_analysis.sql  
    └─ 008_court.sql  
    └─ 009_security.sql  
    └─ 010_retention.sql  
    └─ 011_workflow.sql  
    └─ 012_audit.sql  
    └─ 013_indexes.sql  
    └─ 014_seed_data.sql
```

Þetta er betra en eitt risastórt:

```
schema.sql
```

vegna þess að production schema mun þróast.

3. Migration metadata

Við þurfum migration tracking:

```
CREATE TABLE schema_migrations (  
    version VARCHAR(50) PRIMARY KEY,  
    description TEXT,  
    applied_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Migration tool gæti verið:

- Alembic
- Flyway

- Liquibase
- Sqitch

Fyrir FastAPI/Python myndi ég velja **Alembic**.

4. Extensions

001_extensions.sql:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;
```

Síðar, ef við notum PostgreSQL fyrir vector search:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

En ég myndi **ekki** gera **pgvector** að requirement í fyrsta MVP.

5. Database schemas

Ég myndi ekki hafa allt í **public**.

Nota:

```
ilcms
ilcms_audit
Dæmi:
```

```
CREATE SCHEMA ilcms;
CREATE SCHEMA ilcms_audit;
```

Pá:

```
ilcms
├── cases
├── documents
├── evidence
└── ...
```

```
ilcms_audit
└── audit_log
```

Þetta gefur okkur betri security boundary.

6. Database roles

Production database ætti að hafa að minnsta kosti:

ilcms_owner
ilcms_migrator
ilcms_app
ilcms_readonly
ilcms_audit_writer
Hlutverk:

Role	Purpose
ilcms_owner	Schema owner
ilcms_migrator	Migrations
ilcms_app	API
ilcms_readonly	Reporting
ilcms_audit_write r	Audit insertion

Application user ætti **ekki** að vera schema owner.

7. UUID strategy

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Dæmi:

```
CREATE TABLE ilcms.users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    ...  
);
```

8. Users

```
CREATE TABLE ilcms.users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    external_subject VARCHAR(500) NOT NULL UNIQUE,  
  
    username VARCHAR(200),  
    display_name VARCHAR(500) NOT NULL,  
    email CITEXT NOT NULL,  
  
    status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    last_login_at TIMESTAMPTZ,
```

```

        CONSTRAINT users_status_chk
            CHECK (status IN (
                'ACTIVE',
                'DISABLED',
                'SUSPENDED'
            ))
    );

```

9. Cases

```

CREATE TABLE ilcms.cases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_number VARCHAR(50) NOT NULL UNIQUE,

    title VARCHAR(500) NOT NULL,

    description TEXT,

    case_type_id UUID,

    status VARCHAR(50) NOT NULL DEFAULT 'NEW',

    priority VARCHAR(20) NOT NULL DEFAULT 'NORMAL',

    opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    closed_at TIMESTAMPTZ,

    responsible_user_id UUID
        REFERENCES ilcms.users(id),

    confidentiality_level VARCHAR(50)
        NOT NULL DEFAULT 'CONFIDENTIAL',

    retention_policy_id UUID,

    created_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    updated_by UUID
        REFERENCES ilcms.users(id),

```

```

        updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT cases_status_chk
    CHECK (status IN (
        'NEW',
        'OPEN',
        'ON_HOLD',
        'PRE_LITIGATION',
        'LITIGATION',
        'APPEAL',
        'JUDGMENT',
        'CLOSED',
        'ARCHIVED'
    )),

CONSTRAINT cases_priority_chk
    CHECK (priority IN (
        'LOW',
        'NORMAL',
        'HIGH',
        'CRITICAL'
    ))
);

```

10. Case indexes

```

CREATE INDEX idx_cases_status
ON ilcms.cases(status);

CREATE INDEX idx_cases_responsible
ON ilcms.cases(responsible_user_id);

CREATE INDEX idx_cases_opened
ON ilcms.cases(opened_at);

CREATE INDEX idx_cases_priority
ON ilcms.cases(priority);

```

11. Persons

```

CREATE TABLE ilcms.persons (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    first_name VARCHAR(200) NOT NULL,
    middle_name VARCHAR(200),
    last_name VARCHAR(200),

    display_name VARCHAR(500) NOT NULL,

    nationality VARCHAR(100),

    email CTEXT,
    phone VARCHAR(50),
    address TEXT,

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

12. Organizations

```

CREATE TABLE ilcms.organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    name VARCHAR(500) NOT NULL,

    registration_number VARCHAR(50),

    organization_type VARCHAR(100),

    address TEXT,
    email CTEXT,
    phone VARCHAR(50),

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

13. Parties

```

CREATE TABLE ilcms.parties (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID NOT NULL
        REFERENCES ilcms.cases(id),

    person_id UUID

```

```

REFERENCES ilcms.persons(id),

organization_id UUID
REFERENCES ilcms.organizations(id),

role VARCHAR(50) NOT NULL,

legal_representative_id UUID
REFERENCES ilcms.users(id),

created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT parties_subject_chk
CHECK (
    (person_id IS NOT NULL AND organization_id IS
NULL)
    OR
    (person_id IS NULL AND organization_id IS NOT
NULL)
)
);

```

14. Documents

```

CREATE TABLE ilcms.documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID NOT NULL
REFERENCES ilcms.cases(id),

document_number VARCHAR(50),

document_type VARCHAR(100),

title VARCHAR(500) NOT NULL,

original_filename VARCHAR(1000),
canonical_filename VARCHAR(1000),

document_date DATE,
received_date DATE,

author VARCHAR(500),
sender VARCHAR(500),

```

```

recipient VARCHAR(500),

source VARCHAR(500),

mime_type VARCHAR(200),
file_size BIGINT,

status VARCHAR(50)
    NOT NULL DEFAULT 'UPLOADED',

ocr_status VARCHAR(50)
    NOT NULL DEFAULT 'NOT_REQUIRED',

confidentiality_level VARCHAR(50)
    NOT NULL DEFAULT 'CONFIDENTIAL',

legal_privilege BOOLEAN
    NOT NULL DEFAULT false,

current_version_id UUID,

created_by UUID NOT NULL
    REFERENCES ilcms.users(id),

created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

updated_by UUID
    REFERENCES ilcms.users(id),

updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT documents_status_chk
    CHECK (status IN (
        'UPLOADED',
        'SCANNING',
        'QUARANTINED',
        'VALIDATED',
        'PROCESSING',
        'REVIEW',
        'APPROVED',
        'ACTIVE',
        'REJECTED',
        'ARCHIVED'
    ))
);

```

15. Document versions

```
CREATE TABLE ilcms.document_versions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  document_id UUID NOT NULL  
    REFERENCES ilcms.documents(id),  
  
  version_number INTEGER NOT NULL,  
  
  object_storage_key VARCHAR(2000) NOT NULL,  
  
  sha256 CHAR(64) NOT NULL,  
  
  file_size BIGINT NOT NULL,  
  
  mime_type VARCHAR(200) NOT NULL,  
  
  is_original BOOLEAN NOT NULL DEFAULT false,  
  
  change_reason TEXT,  
  
  created_by UUID NOT NULL  
    REFERENCES ilcms.users(id),  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
  CONSTRAINT document_versions_unique  
    UNIQUE(document_id, version_number)  
);
```

16. Circular FK

Við höfum hér circular dependency:

```
documents  
  ↓  
current_version_id  
  ↓  
document_versions  
  ↓  
document_id  
  ↓
```

documents

Þess vegna:

```
ALTER TABLE ilcms.documents
ADD CONSTRAINT documents_current_version_fk
FOREIGN KEY (current_version_id)
REFERENCES ilcms.document_versions(id);
```

eftir að báðar töflurnar hafa verið búnar til.

17. Document status history

```
CREATE TABLE ilcms.document_status_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  document_id UUID NOT NULL
    REFERENCES ilcms.documents(id),

  from_status VARCHAR(50),
  to_status VARCHAR(50) NOT NULL,

  reason TEXT,

  changed_by UUID NOT NULL
    REFERENCES ilcms.users(id),

  changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

18. Evidence

```
CREATE TABLE ilcms.evidence (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  case_id UUID NOT NULL
    REFERENCES ilcms.cases(id),

  document_id UUID
    REFERENCES ilcms.documents(id),

  evidence_type VARCHAR(100),

  description TEXT,
```

```

source VARCHAR(500),

obtained_at TIMESTAMPTZ,

obtained_by UUID
    REFERENCES ilcms.users(id),

custodian VARCHAR(500),

original_format VARCHAR(100),

integrity_status VARCHAR(50)
    NOT NULL DEFAULT 'UNKNOWN',

relevance VARCHAR(50),

admissibility_status VARCHAR(50),

created_by UUID NOT NULL
    REFERENCES ilcms.users(id),

created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

19. Evidence custody

```

CREATE TABLE ilcms.evidence_custody_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    evidence_id UUID NOT NULL
        REFERENCES ilcms.evidence(id),

    event_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),

    action VARCHAR(100) NOT NULL,

    from_user_id UUID
        REFERENCES ilcms.users(id),

    to_user_id UUID
        REFERENCES ilcms.users(id),

    location VARCHAR(500),

```

```
reason TEXT,  
  
sha256 CHAR(64),  
  
notes TEXT,  
  
created_by UUID NOT NULL  
    REFERENCES ilcms.users(id),  
  
created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

20. Legal analysis

```
CREATE TABLE ilcms.issues (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    case_id UUID NOT NULL  
        REFERENCES ilcms.cases(id),  
  
    title VARCHAR(500) NOT NULL,  
  
    description TEXT,  
  
    status VARCHAR(50)  
        NOT NULL DEFAULT 'OPEN',  
  
    priority VARCHAR(20)  
        NOT NULL DEFAULT 'NORMAL',  
  
    created_by UUID NOT NULL  
        REFERENCES ilcms.users(id),  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    resolved_at TIMESTAMPTZ  
);
```

21. Facts

```
CREATE TABLE ilcms.facts (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    case_id UUID NOT NULL
```

```

        REFERENCES ilcms.cases(id),

statement TEXT NOT NULL,

status VARCHAR(50)
    NOT NULL DEFAULT 'UNKNOWN',

importance VARCHAR(20)
    NOT NULL DEFAULT 'NORMAL',

created_by UUID NOT NULL
    REFERENCES ilcms.users(id),

created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

22. Arguments

```

CREATE TABLE ilcms.legal_arguments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID NOT NULL
        REFERENCES ilcms.cases(id),

    issue_id UUID
        REFERENCES ilcms.issues(id),

    title VARCHAR(500) NOT NULL,

    argument_text TEXT NOT NULL,

    position VARCHAR(50),

    status VARCHAR(50)
        NOT NULL DEFAULT 'DRAFT',

    created_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    updated_by UUID
        REFERENCES ilcms.users(id),

```

```
        updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

23. Legal authorities

```
CREATE TABLE ilcms.legal_authorities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    authority_type VARCHAR(50) NOT NULL,

    title VARCHAR(1000) NOT NULL,

    citation VARCHAR(1000),

    authority_date DATE,

    court VARCHAR(500),

    reference VARCHAR(1000),

    url VARCHAR(2000),

    summary TEXT,

    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

24. Many-to-many tables

```
CREATE TABLE ilcms.fact_evidence (
    fact_id UUID NOT NULL
        REFERENCES ilcms.facts(id),

    evidence_id UUID NOT NULL
        REFERENCES ilcms.evidence(id),

    PRIMARY KEY(fact_id, evidence_id)
);
CREATE TABLE ilcms.argument_facts (
    argument_id UUID NOT NULL
        REFERENCES ilcms.legal_arguments(id),

    fact_id UUID NOT NULL
        REFERENCES ilcms.facts(id),
```

```

        PRIMARY KEY(argument_id, fact_id)
    );
CREATE TABLE ilcms.argument_authorities (
    argument_id UUID NOT NULL
        REFERENCES ilcms.legal_arguments(id),

    authority_id UUID NOT NULL
        REFERENCES ilcms.legal_authorities(id),

    PRIMARY KEY(argument_id, authority_id)
);

```

25. Court

```

CREATE TABLE ilcms.court_proceedings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID NOT NULL
        REFERENCES ilcms.cases(id),

    court VARCHAR(500) NOT NULL,

    court_case_number VARCHAR(100),

    court_level VARCHAR(100),

    jurisdiction VARCHAR(200),

    filing_date DATE,

    status VARCHAR(50),

    judge VARCHAR(500),

    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

26. Court filings

```

CREATE TABLE ilcms.court_filings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    proceeding_id UUID NOT NULL

```

```

        REFERENCES ilcms.court_proceedings(id),

document_id UUID NOT NULL
        REFERENCES ilcms.documents(id),

filing_type VARCHAR(100) NOT NULL,

filing_date TIMESTAMPTZ,

submitted_by UUID
        REFERENCES ilcms.users(id),

submitted_to VARCHAR(500),

submission_method VARCHAR(100),

receipt_timestamp TIMESTAMPTZ,

status VARCHAR(50)
        NOT NULL DEFAULT 'DRAFT',

created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

27. Filing status history

```

CREATE TABLE ilcms.filing_status_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    filing_id UUID NOT NULL
        REFERENCES ilcms.court_filings(id),

    from_status VARCHAR(50),

    to_status VARCHAR(50) NOT NULL,

    reason TEXT,

    changed_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

28. Deadlines

```
CREATE TABLE ilcms.deadlines (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    case_id UUID NOT NULL  
        REFERENCES ilcms.cases(id),  
  
    proceeding_id UUID  
        REFERENCES ilcms.court_proceedings(id),  
  
    title VARCHAR(500) NOT NULL,  
  
    description TEXT,  
  
    due_at TIMESTAMPTZ NOT NULL,  
  
    responsible_user_id UUID  
        REFERENCES ilcms.users(id),  
  
    status VARCHAR(50)  
        NOT NULL DEFAULT 'OPEN',  
  
    source VARCHAR(500),  
  
    reminder_at TIMESTAMPTZ,  
  
    created_by UUID NOT NULL  
        REFERENCES ilcms.users(id),  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

29. Legal holds

```
CREATE TABLE ilcms.legal_holds (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    case_id UUID NOT NULL  
        REFERENCES ilcms.cases(id),  
  
    reason TEXT NOT NULL,  
  
    status VARCHAR(30)
```

```

        NOT NULL DEFAULT 'ACTIVE',

created_by UUID NOT NULL
        REFERENCES ilcms.users(id),

created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

released_by UUID
        REFERENCES ilcms.users(id),

released_at TIMESTAMPTZ
);

```

30. Retention

```

CREATE TABLE ilcms.retention_policies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    name VARCHAR(500) NOT NULL,

    retention_period_days INTEGER,

    trigger_event VARCHAR(100),

    approval_required BOOLEAN NOT NULL DEFAULT true,

    legal_hold_required BOOLEAN NOT NULL DEFAULT true,

    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

31. Retention candidates

```

CREATE TABLE ilcms.retention_candidates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID
        REFERENCES ilcms.cases(id),

    document_id UUID
        REFERENCES ilcms.documents(id),

    eligible_at TIMESTAMPTZ NOT NULL,

```

```

    status VARCHAR(50)
        NOT NULL DEFAULT 'PENDING',

    evaluated_at TIMESTAMPTZ,

    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

32. Disposal requests

```

CREATE TABLE ilcms.disposal_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    retention_candidate_id UUID NOT NULL
        REFERENCES ilcms.retention_candidates(id),

    requested_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    requested_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    approved_by UUID
        REFERENCES ilcms.users(id),

    approved_at TIMESTAMPTZ,

    status VARCHAR(50)
        NOT NULL DEFAULT 'PENDING',

    reason TEXT
);

```

33. Roles

```

CREATE TABLE ilcms.roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    name VARCHAR(100) NOT NULL UNIQUE,

    description TEXT
);

```

Seed:

```
SYSTEM_ADMIN
```

CASE_MANAGER
LAWYER
PARALEGAL
REVIEWER
EXTERNAL_COUNSEL
READ_ONLY
AUDITOR

34. Permissions

```
CREATE TABLE ilcms.permissions (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    resource VARCHAR(100) NOT NULL,  
  
    action VARCHAR(100) NOT NULL,  
  
    UNIQUE(resource, action)  
);
```

35. Role permissions

Ég myndi bæta við:

```
CREATE TABLE ilcms.role_permissions (  
    role_id UUID NOT NULL  
        REFERENCES ilcms.roles(id),  
  
    permission_id UUID NOT NULL  
        REFERENCES ilcms.permissions(id),  
  
    PRIMARY KEY(role_id, permission_id)  
);
```

Þetta var ekki explicit í v0.3 en er nauðsynlegt fyrir raunverulegt RBAC.

36. User roles

```
CREATE TABLE ilcms.user_roles (  
    user_id UUID NOT NULL  
        REFERENCES ilcms.users(id),  
  
    role_id UUID NOT NULL  
        REFERENCES ilcms.roles(id),
```

```
    PRIMARY KEY(user_id, role_id)
);
```

37. Case permissions

```
CREATE TABLE ilcms.case_permissions (
    case_id UUID NOT NULL
        REFERENCES ilcms.cases(id),

    user_id UUID NOT NULL
        REFERENCES ilcms.users(id),

    permission VARCHAR(100) NOT NULL,

    granted_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    granted_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    PRIMARY KEY(case_id, user_id, permission)
);
```

38. Case status history

```
CREATE TABLE ilcms.case_status_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    case_id UUID NOT NULL
        REFERENCES ilcms.cases(id),

    from_status VARCHAR(50),

    to_status VARCHAR(50) NOT NULL,

    reason TEXT,

    changed_by UUID NOT NULL
        REFERENCES ilcms.users(id),

    changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

39. Approval requests

```
CREATE TABLE ilcms.approval_requests (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    case_id UUID NOT NULL  
        REFERENCES ilcms.cases(id),  
  
    resource_type VARCHAR(100) NOT NULL,  
  
    resource_id UUID NOT NULL,  
  
    requested_by UUID NOT NULL  
        REFERENCES ilcms.users(id),  
  
    requested_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    approved_by UUID  
        REFERENCES ilcms.users(id),  
  
    approved_at TIMESTAMPTZ,  
  
    rejected_by UUID  
        REFERENCES ilcms.users(id),  
  
    rejected_at TIMESTAMPTZ,  
  
    status VARCHAR(50)  
        NOT NULL DEFAULT 'PENDING',  
  
    comments TEXT  
);
```

40. Outbox

```
CREATE TABLE ilcms.outbox_events (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    event_type VARCHAR(200) NOT NULL,  
  
    aggregate_type VARCHAR(100) NOT NULL,  
  
    aggregate_id UUID NOT NULL,
```

```
    payload JSONB NOT NULL,  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    processed_at TIMESTAMPTZ  
);
```

Index:

```
CREATE INDEX idx_outbox_unprocessed  
ON ilcms.outbox_events(created_at)  
WHERE processed_at IS NULL;
```

41. Audit schema

Audit fer í sér schema:

```
CREATE TABLE ilcms_audit.audit_log (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    event_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    user_id UUID,  
  
    action VARCHAR(100) NOT NULL,  
  
    entity_type VARCHAR(100) NOT NULL,  
  
    entity_id UUID,  
  
    old_value JSONB,  
  
    new_value JSONB,  
  
    ip_address INET,  
  
    reason TEXT  
);
```

Indexes:

```
CREATE INDEX idx_audit_entity  
ON ilcms_audit.audit_log(entity_type, entity_id);  
  
CREATE INDEX idx_audit_timestamp  
ON ilcms_audit.audit_log(event_timestamp);
```

```
CREATE INDEX idx_audit_user
ON ilcms_audit.audit_log(user_id);
```

42. Audit immutability

Þetta er eitt af mikilvægustu production requirements.

Application role:

```
REVOKE UPDATE, DELETE
ON ilcms_audit.audit_log
FROM ilcms_app;
```

Og:

```
GRANT INSERT
ON ilcms_audit.audit_log
TO ilcms_audit_writer;
```

Ekki treysta eingöngu á application code til að tryggja þetta.

43. updated_at trigger

Sameiginleg function:

```
CREATE OR REPLACE FUNCTION ilcms.set_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$;
```

Síðan:

```
CREATE TRIGGER trg_cases_updated_at
BEFORE UPDATE ON ilcms.cases
FOR EACH ROW
EXECUTE FUNCTION ilcms.set_updated_at();
```

Sama pattern fyrir:

```
persons
organizations
documents
legal_arguments
```

44. Case state machine

Ég myndi ekki leyfa arbitrary status updates.

Trigger/function:

NEW → OPEN
OPEN → PRE_LITIGATION
PRE_LITIGATION → LITIGATION
LITIGATION → APPEAL

...

Ef einhver reynir:

ARCHIVED → OPEN

án sérstaks workflow:

ERROR

Þetta ætti helst að vera **service-layer authorization + database constraint/trigger fyrir critical invariants**.

45. Important architectural decision

Ekki setja **alla business logic** í PostgreSQL triggers.

Betra:

API/service layer

↓

Business workflow

↓

PostgreSQL

↓

Critical invariants

Database skal tryggja:

- FK integrity
- uniqueness
- basic state invariants
- append-only properties
- critical security boundaries

en application layer sér um flóknari workflow.

46. Document integrity

Við upload:

binary

↓

SHA-256

↓

document_versions.sha256

Við download/export getum við sannreynt:

stored SHA-256

==

calculated SHA-256

47. Object storage model

PostgreSQL geymir:

document_versions.object_storage_key

en binary fer í:

Object Storage

Dæmi:

cases/

 {case_uuid}/

 documents/

 {document_uuid}/

 {version_uuid}

Ekki nota original filename sem storage key.

48. Database → Object storage consistency

Þetta er edge case sem þarf að hanna frá byrjun.

Dæmi:

1. Upload binary
2. Calculate SHA256
3. Store object
4. BEGIN DB transaction
5. Create document
6. Create version
7. Create outbox event

8. COMMIT

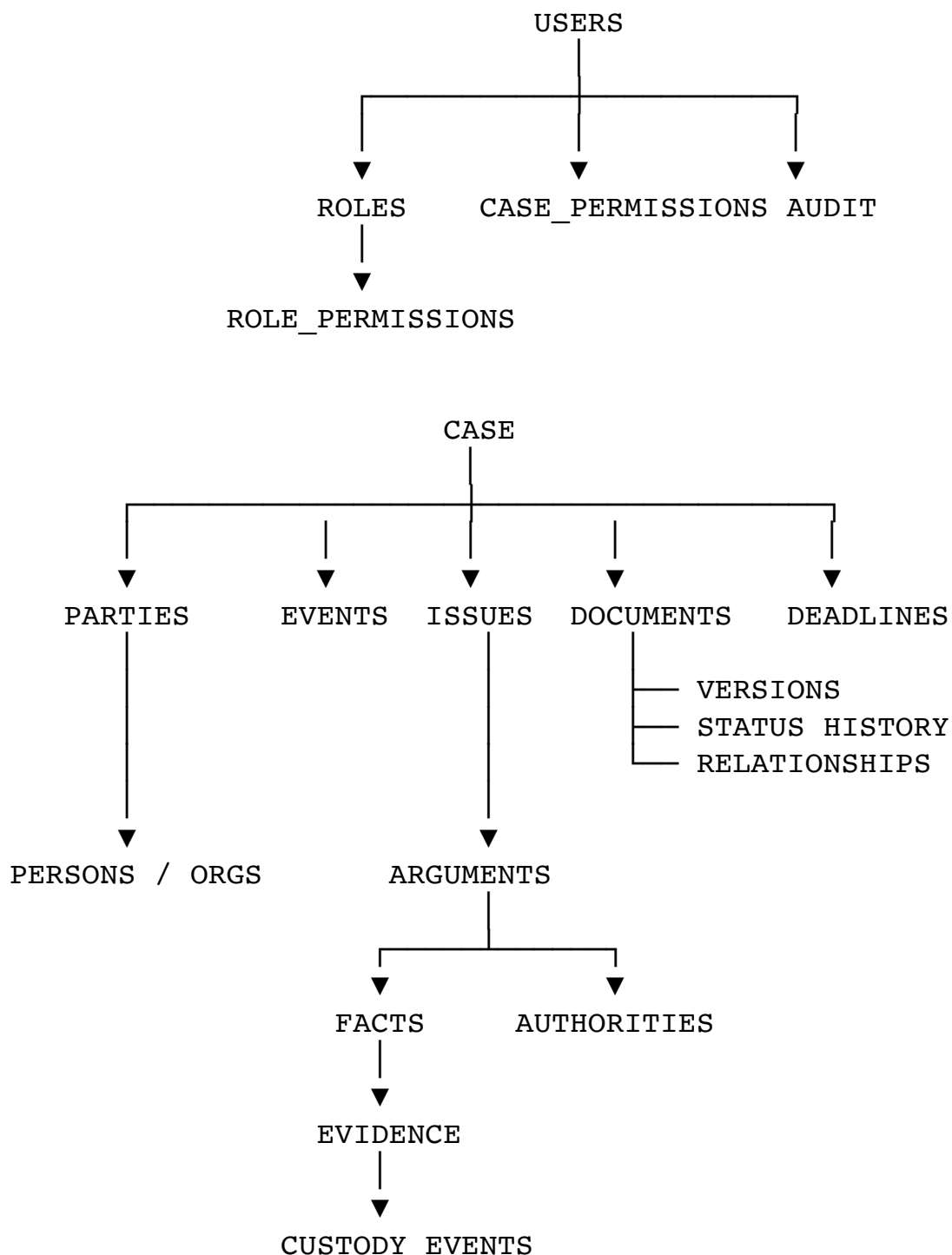
Ef DB transaction mistekst:

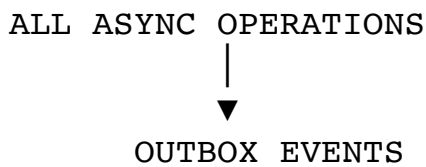
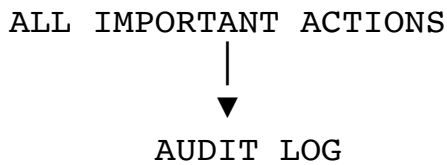
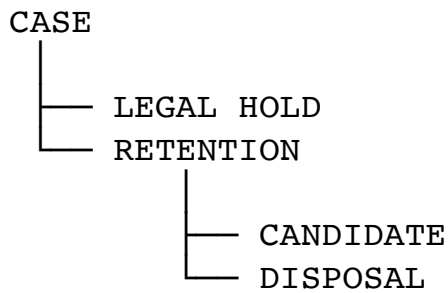
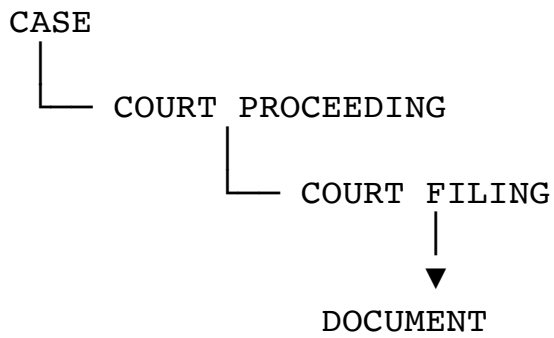
orphan object

Worker þarf að geta fundið og hreinsað orphan objects.

49. ER model — production

Heildar gagnalíkanið er nú:





50. Seed permissions

Ég myndi byrja með eftirfarandi permission vocabulary:

CASE:CREATE
CASE:READ
CASE:UPDATE
CASE:CLOSE
CASE:ARCHIVE

PARTY:READ
PARTY:WRITE

DOCUMENT:READ

DOCUMENT:CREATE
DOCUMENT:UPDATE
DOCUMENT:DOWNLOAD
DOCUMENT:EXPORT
DOCUMENT:APPROVE

EVIDENCE:READ
EVIDENCE:CREATE
EVIDENCE:TRANSFER

ISSUE:READ
ISSUE:WRITE

FACT:READ
FACT:WRITE

ARGUMENT:READ
ARGUMENT:WRITE

FILING:READ
FILING:CREATE
FILING:REVIEW
FILING:APPROVE
FILING:SUBMIT

DEADLINE:READ
DEADLINE:WRITE

LEGAL_HOLD:CREATE
LEGAL_HOLD:RELEASE

RETENTION:READ
RETENTION:APPROVE
RETENTION:DISPOSE

AUDIT:READ

USER:READ
USER:ADMIN

51. Role seed example

Til dæmis:

LAWYER

- CASE:CREATE
- CASE:READ
- CASE:UPDATE
- DOCUMENT:READ
- DOCUMENT:CREATE
- DOCUMENT:DOWNLOAD
- EVIDENCE:READ
- EVIDENCE:CREATE
- ISSUE:READ
- ISSUE:WRITE
- FACT:READ
- FACT:WRITE
- ARGUMENT:READ
- ARGUMENT:WRITE
- FILING:CREATE

FILING:APPROVE ætti ekki sjálfkrafa að fylgja FILING:CREATE.

52. Indexing strategy

Fyrsta production indexing:

cases

- status
- responsible_user_id
- opened_at
- priority

documents

- case_id
- status
- document_date
- created_at

document_versions

- document_id
- sha256

evidence

- case_id
- document_id

events

- case_id + event_date

deadlines
├─ case_id
├─ responsible_user_id
└─ due_at

audit
├─ entity
├─ user
└─ timestamp

Ekki index-a allt. Of mörg indexes hægja á writes.

53. Full-text search

MVP:

PostgreSQL
↓
tsvector
Production scale:

PostgreSQL
└─ Outbox
 ↓
 OpenSearch

Search index ætti **aldrei** að vera authoritative source.

54. Backup strategy

Production PostgreSQL:

Continuous WAL archiving
+
Daily backup
+
Periodic full snapshot
Markmið:

$RPO \leq 15 \text{ min}$

$RTO \leq 1 \text{ hour}$

fyrir fyrstu production útgáfu.

Þetta eru **design targets**, ekki lagaleg krafa.

55. Encryption

At rest

PostgreSQL disk
Object storage
Backup
allt encrypted.

In transit

Client → API = TLS
API → PostgreSQL = TLS
API → Object Storage = TLS
Workers → Services = TLS

Application secrets

Ekki:

DATABASE_PASSWORD=...
í Git.

Nota secret management:

Vault
AWS Secrets Manager
Azure Key Vault
Kubernetes Secrets + KMS

56. Row Level Security

Ég myndi **meta RLS** fyrir `cases` og `documents`.

Dæmi:

```
ALTER TABLE ilcms.cases  
ENABLE ROW LEVEL SECURITY;
```

Þetta gæti gefið okkur:

PostgreSQL

↓

RLS

↓

Case ACL

En ég myndi ekki gera RLS endanlegt fyrr en identity propagation og connection pooling strategy eru ákveðin.

v0.5 decision: RLS er architectural option, ekki mandatory MVP requirement.

57. API mapping

Database resources tengjast REST API:

<code>cases</code>	→ <code>/cases</code>
<code>parties</code>	→ <code>/cases/{id}/parties</code>
<code>documents</code>	→ <code>/documents</code>
<code>document_versions</code>	→ <code>/documents/{id}/versions</code>
<code>evidence</code>	→ <code>/evidence</code>
<code>events</code>	→ <code>/cases/{id}/events</code>
<code>issues</code>	→ <code>/cases/{id}/issues</code>
<code>facts</code>	→ <code>/cases/{id}/facts</code>
<code>arguments</code>	→ <code>/cases/{id}/arguments</code>
<code>proceedings</code>	→ <code>/cases/{id}/proceedings</code>
<code>filings</code>	→ <code>/proceedings/{id}/filings</code>
<code>deadlines</code>	→ <code>/cases/{id}/deadlines</code>

Internal tables eins og:

`audit_log`
`outbox_events`
`retention_candidates`
verða ekki almennt exposed beint.

58. Transaction boundary

Dæmi um að stofna case:

```
POST /cases
  |
  ▼
BEGIN
  |
  ├── INSERT cases
  ├── INSERT status_history
  ├── INSERT audit
  └── INSERT outbox_event
  |
  COMMIT
```

Pannig verður case creation atomic.

59. Test requirements

Áður en v0.5 telst production-ready þurfum við database tests fyrir:

Referential integrity

Cannot create document for nonexistent case.

State transitions

Cannot ARCHIVED → OPEN.

Evidence

Cannot modify custody event.

Audit

Application cannot UPDATE audit.

Application cannot DELETE audit.

Legal hold

Active legal hold prevents disposal.

Permissions

User without case access cannot read case.

60. Migration test

CI pipeline:

Clean PostgreSQL

↓

Run migrations

↓

Run seed

↓

Run schema validation

↓

Run integration tests

↓

Rollback test

Ég myndi keyra þetta á hverri breytingu í migration directory.

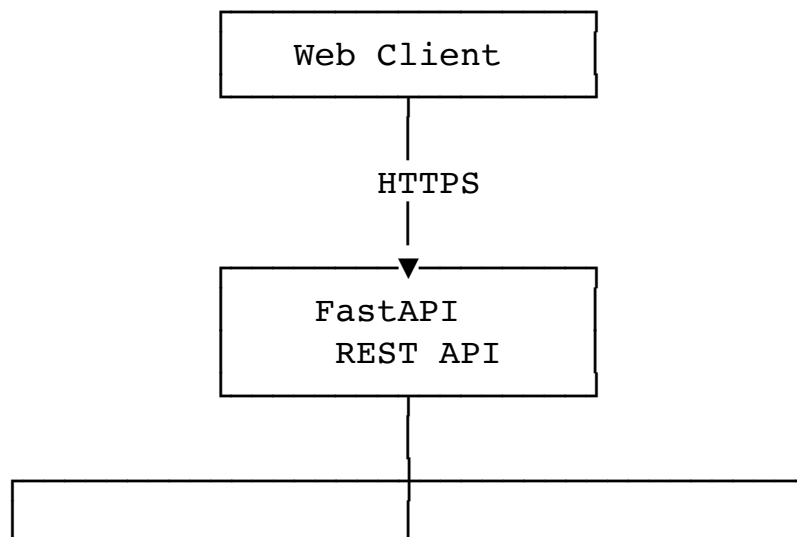
61. Recommended repository structure

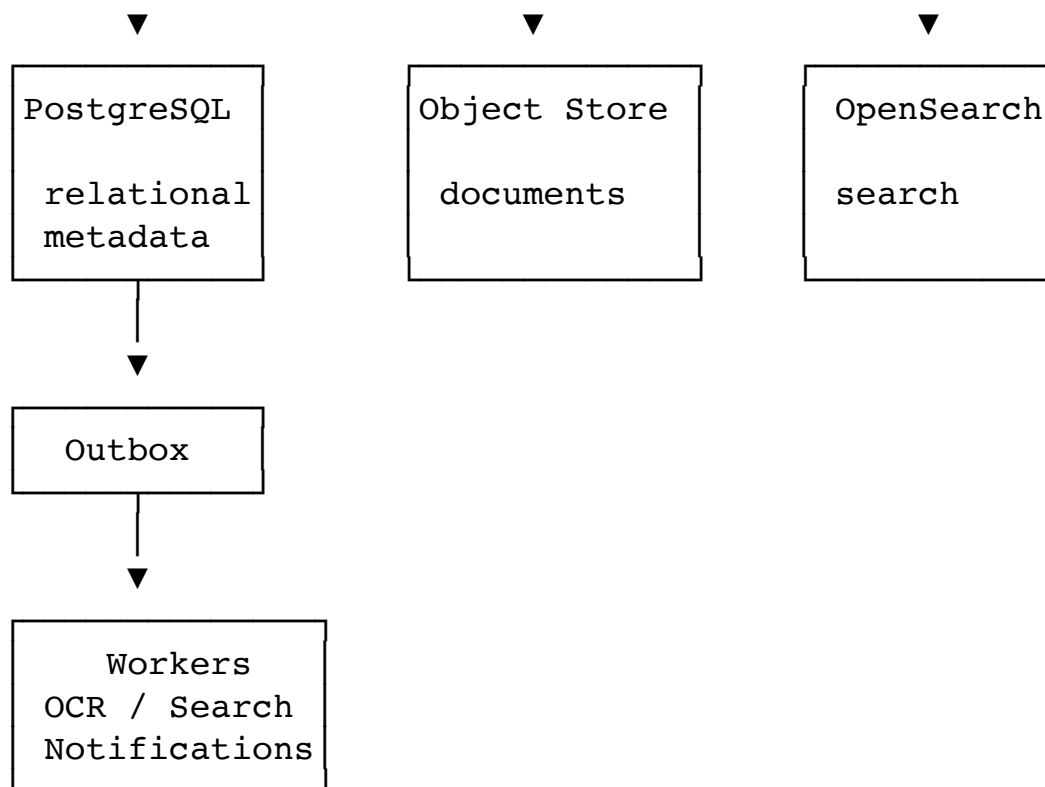
Ef við byggjum þetta í Python/FastAPI:

```
ilcms/  
├── app/  
│   ├── api/  
│   ├── models/  
│   ├── schemas/  
│   ├── services/  
│   ├── security/  
│   ├── workflows/  
│   └── workers/  
├── migrations/  
│   └── versions/  
├── tests/  
│   ├── unit/  
│   ├── integration/  
│   └── security/  
├── docker/  
├── docs/  
├── pyproject.toml  
└── README.md
```

62. v0.5 architectural baseline

Við höfum nú komið með kerfið á þetta stig:





63. Mikilvæg niðurstaða

Við erum nú komin yfir mikilvægan þröskuld.

v0.1–v0.2 voru fyrst og fremst hugmyndafræðileg hönnun.

v0.3–v0.4 skilgreindu API, workflow og security.

v0.5 skilgreinir nú hvernig kerfið getur í raun verið sett upp í PostgreSQL.

Næsta skref ætti því ekki lengur að vera bara meiri gagnagrunnshönnun.

Technical Specification v0.6

Ég myndi leggja til að næsta útgáfa verði:

v0.6 — Application Architecture & Implementation Specification

Þar getum við ákveðið **hvernig allt kerfið verður keyrt**, t.d.:

Frontend

↓

API

↓
Service Layer

↓
Repositories

↓
PostgreSQL
og síðan:

Document Upload

↓
Object Storage

↓
Queue

↓
Malware Scan

↓
OCR

↓
Document Classification

↓
Search Index

Við getum þá valið tæknistakk, t.d.:

- **Frontend:** React + TypeScript
- **Backend:** Python + FastAPI
- **Database:** PostgreSQL
- **Object storage:** S3-compatible
- **Search:** OpenSearch
- **Queue:** RabbitMQ / Redis / cloud queue
- **OCR:** Tesseract eða commercial OCR
- **Identity:** OIDC / Entra ID / Keycloak
- **Deployment:** Docker + Kubernetes
- **IaC:** Terraform
- **Observability:** OpenTelemetry + Prometheus + Grafana

Og mjög áhugavert fyrir þetta verkefni: í **v0.6** ættum við einnig að skilgreina **AI architecture** sem sérstakt security-isolated subsystem — þannig að síðar sé hægt að bæta við t.d. document summarization, legal research, semantic search og case analysis án þess að AI-lagið fái óheftan aðgang að lögfræðilegum gögnum.